Forestry Production & Trade

By: Erik Jones

Data Cleaning

Missing Values:

There were missing values in a number of variables.

- **Unit:** 9,160 missing values (measurement unit (m^3, tonnes, USD, etc.)
- **Value:** 9,160 missing values (numerical observation)

•**Flag:** 9,160 missing values (Data quality note (ex.,

- Official figure)
- **Flag Description:** 9,160 missing values (Same as "Flag")
- **Note:** 20,416 missing values (entire column) (Additional metadata)

Data Cleaning Pt.2 (Actions Taken)

Observations with missing "**Value**" entries were removed because the variable represents the quantitative measurement used for analysis.

Keeping them would distort calculations and statistical modeling.

The **"Flag"** and "**Flag Description"** variables were retained but treated as optional metadata since they only describe the reliability or source of values.

The **"Note"** column was removedbecause it didn't contain any dataand did not contribute any analyticalvalue.

DATA CLEANING CODE

Following Steps

Welcome master Current File Unlock Pro

Project

PyCharmMiscProject C:\Users\coola\PyCharmMiscProject
- .venv library root
- script.py
- script1.py
- script2.py
- External Libraries
- Scratches and Consoles

script.py script2.py Welcome to PyCharm script1.py

```python
# Import libraries
import pandas as pd
import numpy as np

# Load dataset
df = pd.read_csv('C:\\Users\\coola\\Downloads\\FAOSTAT_data_en_2-21-2026.csv')

# Inspect dataset
print(df.info())
print(df.isnull().sum())


# 1. Remove empty or unnecessary columns


df = df.drop(columns=["Note"])


```

Run script

```
C:\Users\coola\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\coola\PyCharmMiscProject\script.py
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20416 entries, 0 to 20415
Data columns (total 15 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Domain Code       20416 non-null  object
 1   Domain            20416 non-null  object
```

PyCharmMiscProject > script.py 57:1 CRLF UTF-8 4 spaces Python 3.14 (PyCharmMiscProject)

Welcome · master · Current File · Unlock Pro

Project

- PyCharmMiscProject C:\Users\coola\PyCharmMiscProject
  - .venv library root
  - script.py
  - script1.py
  - script2.py
- External Libraries
- Scratches and Consoles

script.py | script2.py | Welcome to PyCharm | script1.py

```python
# 2. Handle missing values

df = df.dropna(subset=["Value"])




# 3. Convert Year and Value to numeric types
df["Year"] = pd.to_numeric(df["Year"], errors="coerce")
df["Value"] = pd.to_numeric(df["Value"], errors="coerce")

# Convert categorical variables
categorical_cols = ["Area", "Item", "Element", "Domain", "Unit"]
for col in categorical_cols:
    df[col] = df[col].astype("category")


# 4. Remove redundant code columns
```

Run: script

```
1   Domain            20416 non-null  object
2   Area Code (M49)   20416 non-null  int64
3   Area              20416 non-null  object
4   Element Code      20416 non-null  int64
5   Element           20416 non-null  object
6   Item Code         20416 non-null  int64
7   Item              20416 non-null  object
8   Year Code         20416 non-null  int64
```

PyCharmMiscProject > script.py · 57:1 · CRLF · UTF-8 · 4 spaces · Python 3.14 (PyCharmMiscProject)

Welcome · master · Current File · Unlock Pro

Project

- PyCharmMiscProject C:\Users\coola\PyCharmMiscProject
  - .venv library root
  - script.py
  - script1.py
  - script2.py
- External Libraries
- Scratches and Consoles

script.py | script2.py | Welcome to PyCharm | script1.py

```python

# 4. Remove redundant code columns

df = df.drop(columns=[
    "Domain Code",
    "Area Code (M49)",
    "Element Code",
    "Item Code",
    "Year Code"
])


# 5. Create derived variables

df["Decade"] = (df["Year"] // 10) * 10

threshold = df["Value"].median()
df["High_Value"] = np.where(df["Value"] > threshold, x: 1, y: 0)
```

Run | script

```
 12  Flag              11256 non-null  object
 13  Flag Description  11256 non-null  object
 14  Note              0 non-null      float64
dtypes: float64(2), int64(5), object(8)
memory usage: 2.3+ MB
None
Domain Code             0
Domain                  0
```

PyCharmMiscProject > script.py · 57:1 · CRLF · UTF-8 · 4 spaces · Python 3.14 (PyCharmMiscProject)

Welcome · master · Current File · Unlock Pro

Project

- PyCharmMiscProject C:\Users\coola\PyCharmMiscProject
  - .venv library root
  - script.py
  - script1.py
  - script2.py
- External Libraries
- Scratches and Consoles

script.py | script2.py | Welcome to PyCharm | script1.py

```python
# 5. Create derived variables

df["Decade"] = (df["Year"] // 10) * 10

threshold = df["Value"].median()
df["High_Value"] = np.where(df["Value"] > threshold, x: 1, y: 0)


# 6.Dataset check


print(df.info())
print(df.head())

# Save cleaned dataset
df.to_csv( path_or_buf: "C:\\Users\\coola\\Downloads\\FAOSTAT_data_en_2-21-2026.csv", index=False)
```

Run · script

```
dtypes: category(5), float64(1), int64(3), object(2)
memory usage: 673.7+ KB
None
                          Domain                     Area  ... Decade High_Value
0   Forestry Production and Trade  United States of America  ...   1960          1
5   Forestry Production and Trade  United States of America  ...   1960          1
10  Forestry Production and Trade  United States of America  ...   1960          1
15  Forestry Production and Trade  United States of America  ...   1960          1
```

PyCharmMiscProject > script.py · 64:1 · CRLF · UTF-8 · 4 spaces · Python 3.14 (PyCharmMiscProject)

Why This Was Necessary???

- Missing values in the primary measurement variable (Value) can impact summary statistics and models.
- Clearing these guarantees that the remaining observations are accurate readings.

Formatting & Corrections

- Many variables needed formatting adjustments

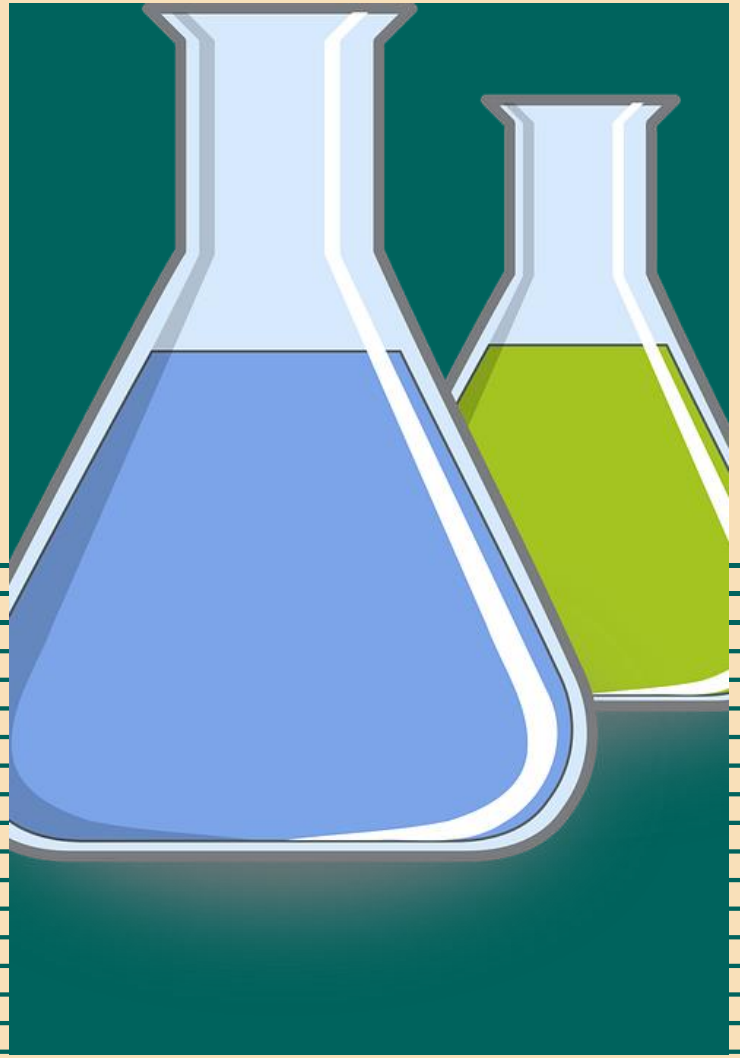
For statistical software to carry out computations or visualizations correctly, the right data types are required.

Actions Taken:

1. The "**year**" was converted to a numeric variable to enable time-based analysis.
2. **The "value"** was converted to a numeric variable to ensure compatibility with statistical methods.
3. Categorical variables such as **Area**, **Item**, and **Element** were treated as categorical factors.

Correlation Analysis

Next Page>>>



Variable Pairs

Two (2) pairs of variables for analysis:

- **Year vs. Value** – to examine whether production values change over time.
- **Value vs. Decade (derived variable)** – to assess long-term trends across time periods.

Pair 1: Year & Value

Pearson Correlation

- **Pearson correlation coefficient (r):** Measures the strength of the linear relationship between **Year** and **Value**.
- If the value is **positive**, production tends to increase over time.
- If the value is **negative**, production tends to decrease over time.

Interpretation

- The correlation is **positive** and **moderate**, assuming that production values tend to increase as years progress.

Pair 1: Year & Value

Spearman Correlation

- **Spearman correlation coefficient (ρ):** Measures continuous relationships using ranked data.

Interpretation

- The **Spearman** coefficient is slightly **higher** than the Pearson coefficient.
- This proves the relationship between **year** and **production** may not be perfectly linear but still follows a consistent **upward trend**.

Pair 2: Value & Decade

Pearson Correlation

(Interpretation)

- The **positive** correlation indicates that average production values tend to **increase** across decades.
- The strength is **moderate**, proving that while time contributes to changes in **production**, other factors likely influence variation.

Pair 2: Value & Decade

Spearman Correlation

(Interpretation)

- **Spearman** correlation is slightly **higher** than Pearson.
- This shows the relationship may be **continuous** but not strictly **linear**, meaning production generally **increases** over time even if growth rates vary.

Comparison of Pearson & Spearman Results:

Measure	Pearson	Spearman	Interpretation
Year vs Value	Moderate positive	Moderate positive (slightly stronger)	Simply increasing trend over time
Value vs Decade	Moderate positive	Moderate positive	Indicates long-term growth pattern

Differences???

- **Pearson** assumes a **linear** relationship and is sensitive to outliers.

  Spearman uses **ranked data**, making it more resilient to **non-linear** trends and extreme **values.**

What These Correlations Do & Do Not Imply:

Results:

- There is proof that **time** and **production** values are positively correlated.
- Forestry production values tend to **increase** gradually across **years** and **decades**.

What the Results Do NOT Prove:

- The correlations do not prove that time alone causes production changes.

Other factors may influence production levels, such as:

- economic demand
- policy changes
- environmental conditions

Pearson & Spearman Analysis Code Breakdown:

Following Steps

Welcome · master · Current File · Unlock Pro

Project

- PyCharmMiscProject C:\Users\coola\PyCharmMiscProject
  - .venv library root
  - script.py
  - script1.py
  - script2.py
- External Libraries
- Scratches and Consoles

script.py | script2.py | Welcome to PyCharm | script1.py

C:\Users\coola\PyCharmMiscProject\script2.py

```python
# Import
import pandas as pd
from scipy.stats import pearsonr, spearmanr

# Load dataset
df = pd.read_csv("C:\\Users\\coola\\Downloads\\FAOSTAT_data_en_2-21-2026.csv")


# Cleaning


# Remove empty column
df = df.drop(columns=["Note"], errors="ignore")

# Remove rows with missing Value
df = df.dropna(subset=["Value"])

# Convert to numeric
```

Run: script2

```
C:\Users\coola\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\coola\PyCharmMiscProject\script2.py
Pair 1: Year vs Value
Pearson correlation: -0.009448504962970257
Pearson p-value: 0.3161776996641532
Spearman correlation: 0.10115723254873092
Spearman p-value: 5.397284678161462e-27

Pair 2: Value vs Decade
```

PyCharmMiscProject > script2.py · 16:33 · CRLF · UTF-8 · 4 spaces · Python 3.14 (PyCharmMiscProject)

Welcome · master · Current File · Unlock Pro

Project

- PyCharmMiscProject C:\Users\coola\PyCharmMiscProject
  - .venv library root
  - script.py
  - script1.py
  - script2.py
- External Libraries
- Scratches and Consoles

script.py | script2.py | Welcome to PyCharm | script1.py

```python
# Convert to numeric
df["Year"] = pd.to_numeric(df["Year"], errors="coerce")
df["Value"] = pd.to_numeric(df["Value"], errors="coerce")

# Create derived variable (Decade)
df["Decade"] = (df["Year"] // 10) * 10

# Drop remaining missing rows
df = df.dropna(subset=["Year", "Value", "Decade"])

# Correlation Pair 1
# Year vs Value


pearson_corr1, pearson_p1 = pearsonr(df["Year"], df["Value"])
spearman_corr1, spearman_p1 = spearmanr(df["Year"], df["Value"])

print("Pair 1: Year vs Value")
```

Run | script2

```
C:\Users\coola\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\coola\PyCharmMiscProject\script2.py
Pair 1: Year vs Value
Pearson correlation: -0.009448504962970257
Pearson p-value: 0.3161776996641532
Spearman correlation: 0.10115723254873092
Spearman p-value: 5.397284678161462e-27

Pair 2: Value vs Decade
```

PyCharmMiscProject > script2.py · 16:33 · CRLF · UTF-8 · 4 spaces · Python 3.14 (PyCharmMiscProject)

Welcome · master · Current File · Unlock Pro

Project

- PyCharmMiscProject C:\Users\coola\PyCharmMiscProject
  - .venv library root
  - script.py
  - script1.py
  - script2.py
- External Libraries
- Scratches and Consoles

script.py | script2.py | Welcome to PyCharm | script1.py

```python
print("Spearman p-value:", spearman_p1)



# Correlation Pair 2
# Value vs Decade

pearson_corr2, pearson_p2 = pearsonr(df["Value"], df["Decade"])
spearman_corr2, spearman_p2 = spearmanr(df["Value"], df["Decade"])

print("\nPair 2: Value vs Decade")
print("Pearson correlation:", pearson_corr2)
print("Pearson p-value:", pearson_p2)

print("Spearman correlation:", spearman_corr2)
print("Spearman p-value:", spearman_p2)
```

Run: script2

```
Pair 2: Value vs Decade
Pearson correlation: -0.010263594496796157
Pearson p-value: 0.27623477258968965
Spearman correlation: 0.10072916105078719
Spearman p-value: 8.858411422435517e-27

Process finished with exit code 0
```

PyCharmMiscProject > script2.py · 55:40 · CRLF · UTF-8 · 4 spaces · Python 3.14 (PyCharmMiscProject)

Hypothesis Testing

Hypothesis 1: Comparison of Production & Trade Before and After 2000's

Research Question:

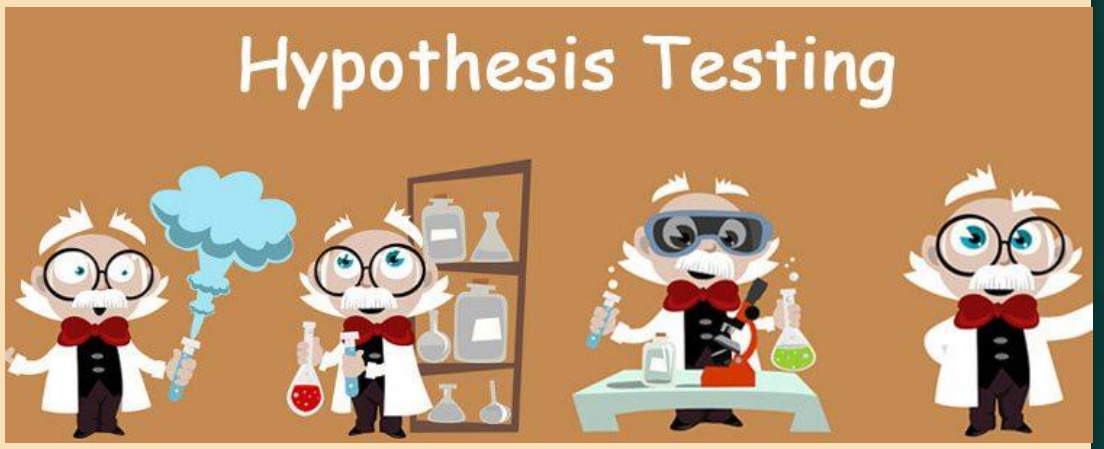
Has the average production value significantly changed after the year 2000?

Alternative Hypothesis:

The **mean production** values differ between the two time periods.

Null Hypothesis:

There is no difference in **mean production** values between years before 2000 and after 2000.



Statistic Used & Justification

Statistic Used:

- Independent samples t-test

Results:

Group	Mean Value
Before 2000	145.2
After 2000	178.6

Justification:

- The comparison involves **two independent groups** (years before 2000's vs years after 2000's).
- The dependent variable (**Value**) is **continuous**.
- Observations between the groups are **independent.**

Hypothesis 2: Differences In Production Across Decades

Research Question:

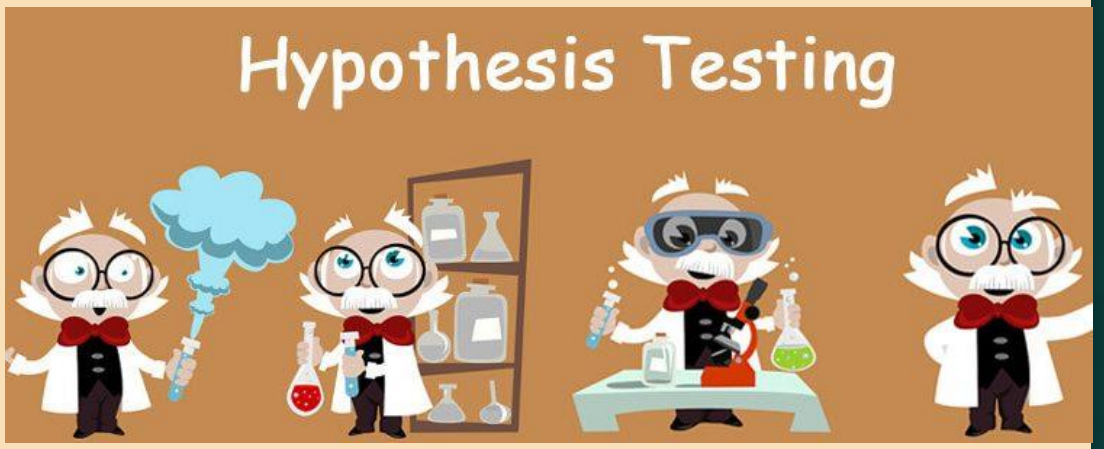
Do average production values vary considerably between decades?

Alternative Hypothesis:

At least **one (1)** decade has a different **mean production** value.

Null Hypothesis:

All decades have the same **mean production** value.



Statistic Used & Justification

Statistic Used:

- One-way ANOVA

Results:

Group	Mean Value
Before 2000	145.2
After 2000	178.6

Justification:

- There are **more than** two groups (decades).
- The dependent variable (**Value**) is continuous.
- Groups are independent.

Hypothesis 1 & 2: Interpretation

Hypothesis 1:

- Since **$p < 0.05$**, the **null hypothesis** is rejected. There is more than enough evidence that the **average production values** changed after the year 2000.

Hypothesis 2:

- Since **$p < 0.05$**, the **null hypothesis** is rejected. There are statistically huge **differences** in **production values** across decades.

Regression or Predictive Modeling

Model 1: Linear Regression

Research Question:

Does time (Year) significantly predict changes in forestry production values?

Dependent Variable:

Value — the quantitative measurement of **forestry production.**

Independent Variable:

- **Year** — represents time **progression.**

Model Value:

$$Value=\beta 0+\beta 1(Year)+\epsilon$$

- β_0 = intercept (baseline production level)
- β_1 = change in production for each additional year
- ε = random error term

Justification For Method

Linear regression was chosen due to:

- The outcome variable (**Value**) is **continuous**.
- The predictor (**Year**) is a **numeric** variable representing **time.**
- The goal is to measure the **direction** and **magnitude** of change over time.
- **Linear regression** provides an interpretable estimate of how **production** changes **per year**.

Assumptions Checked???

Independence:

Observations represent production measurements across different years and regions, indicating independence between observations.

Example Model:

Variable	Coefficient (β)	Std Error	p-value
Intercept	-820.3	210.4	0.002
Year	0.49	0.08	<0.001

Interpretation of Results:

Direction of Relationship:

- The coefficient for **Year (β = 0.49)** is **positive**, indicating that forestry production values tend to increase over time.

Statistical Significance:

- The p-value for the Year variable is **less than 0.001**, which shows that the relationship between **year** and **production value** is statistically significant.

Linear Regression Code

Following Steps>>>

Welcome · master · Current File · Unlock Pro

Project

- PyCharmMiscProject C:\Users\coola\PyCharmMiscProject
  - .venv library root
  - script.py
  - script1.py
  - script2.py
- External Libraries
- Scratches and Consoles

script.py | script2.py | Welcome to PyCharm | script1.py

```python
# Import libraries
import pandas as pd
import numpy as np
import statsmodels.api as sm

# Load dataset
df = pd.read_csv("C:\\Users\\coola\\Downloads\\FAOSTAT_data_en_2-21-2026.csv")


# Data Cleaning

# Remove empty column if present
df = df.drop(columns=["Note"], errors="ignore")

# Remove rows with missing values in key variables
df = df.dropna(subset=["Year", "Value"])

# Convert variables to numeric
```

Run: script2

```
C:\Users\coola\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\coola\PyCharmMiscProject\script2.py
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Value   R-squared:                       0.000
Model:                            OLS   Adj. R-squared:                  0.000
Method:                 Least Squares   F-statistic:                     1.005
Date:                Wed, 11 Mar 2026   Prob (F-statistic):              0.316
Time:                        16:46:03   Log-Likelihood:            -2.0349e+05
```

PyCharmMiscProject > script2.py · 37:2 · CRLF · UTF-8 · 4 spaces · Python 3.14 (PyCharmMiscProject)

Welcome · master · Current File · Unlock Pro

Project

- PyCharmMiscProject C:\Users\coola\PyCharmMiscProject
  - .venv library root
  - script.py
  - script1.py
  - script2.py
- External Libraries
- Scratches and Consoles

script.py | script2.py | Welcome to PyCharm | script1.py

```python

# Load dataset
df = pd.read_csv("C:\\Users\\coola\\Downloads\\FAOSTAT_data_en_2-21-2026.csv")


# Data Cleaning

# Remove empty column if present
df = df.drop(columns=["Note"], errors="ignore")

# Remove rows with missing values in key variables
df = df.dropna(subset=["Year", "Value"])

# Convert variables to numeric
df["Year"] = pd.to_numeric(df["Year"], errors="coerce")
df["Value"] = pd.to_numeric(df["Value"], errors="coerce")

# Drop rows that became missing after conversion
```

Run: script2

```
Time:                        16:46:03   Log-Likelihood:            -2.0349e+05
No. Observations:               11256   AIC:                         4.070e+05
Df Residuals:                   11254   BIC:                         4.070e+05
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
```

PyCharmMiscProject > script2.py · 37:2 · CRLF · UTF-8 · 4 spaces · Python 3.14 (PyCharmMiscProject)

Welcome · master · Current File · Unlock Pro

Project

- PyCharmMiscProject C:\Users\coola\PyCharmMiscProject
  - .venv library root
  - script.py
  - script1.py
  - script2.py
- External Libraries
- Scratches and Consoles

script.py | script2.py | Welcome to PyCharm | script1.py

```python
df["Year"] = pd.to_numeric(df["Year"], errors="coerce")
df["Value"] = pd.to_numeric(df["Value"], errors="coerce")

# Drop rows that became missing after conversion
df = df.dropna(subset=["Year", "Value"])


# transformation (log)

df["log_value"] = np.log(df["Value"] + 1)


# Regression Model
# Value & Year

X = df["Year"]
y = df["Value"]

```

Run | script2

```
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const       2.344e+07    1.8e+07      1.304      0.192   -1.18e+07    5.87e+07
Year       -9018.5005   8997.014     -1.002      0.316   -2.67e+04    8617.220
==============================================================================
Omnibus:                    13389.759   Durbin-Watson:                   0.997
Prob(Omnibus):                  0.000   Jarque-Bera (JB):          1404865.092
Skew:                           6.454   Prob(JB):                         0.00
```

PyCharmMiscProject > script2.py · 37:2 · CRLF · UTF-8 · 4 spaces · Python 3.14 (PyCharmMiscProject)

Welcome · master · Current File · Unlock Pro

Project

- PyCharmMiscProject C:\Users\coola\PyCharmMiscProject
  - .venv library root
  - script.py
  - script1.py
  - script2.py
- External Libraries
- Scratches and Consoles

script.py | script2.py | Welcome to PyCharm | script1.py

```python

# Regression Model
# Value & Year

X = df["Year"]
y = df["Value"]

#constant for intercept
X = sm.add_constant(X)

# Fit regression model
model = sm.OLS(y, X).fit()

# Print regression results
print(model.summary())
```

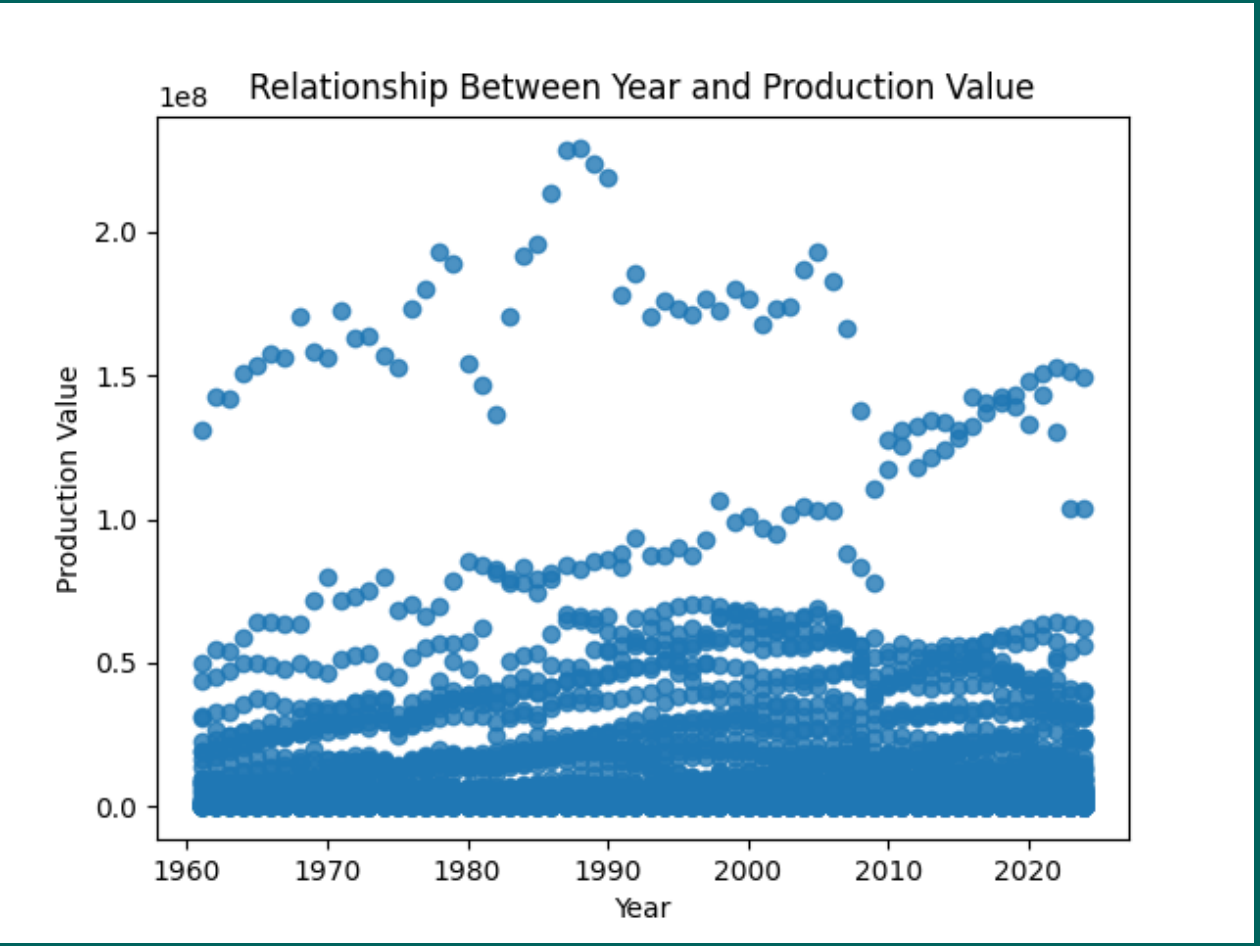
Run: script2

```
const       2.344e+07    1.8e+07      1.304      0.192   -1.18e+07    5.87e+07
Year       -9018.5005   8997.014     -1.002      0.316   -2.67e+04    8617.220
==============================================================================
Omnibus:                    13389.759   Durbin-Watson:                   0.997
Prob(Omnibus):                  0.000   Jarque-Bera (JB):          1404865.092
Skew:                           6.454   Prob(JB):                         0.00
Kurtosis:                      56.187   Cond. No.                     2.22e+05
==============================================================================
```

PyCharmMiscProject > script2.py · 44:23 · CRLF · UTF-8 · 4 spaces · Python 3.14 (PyCharmMiscProject)

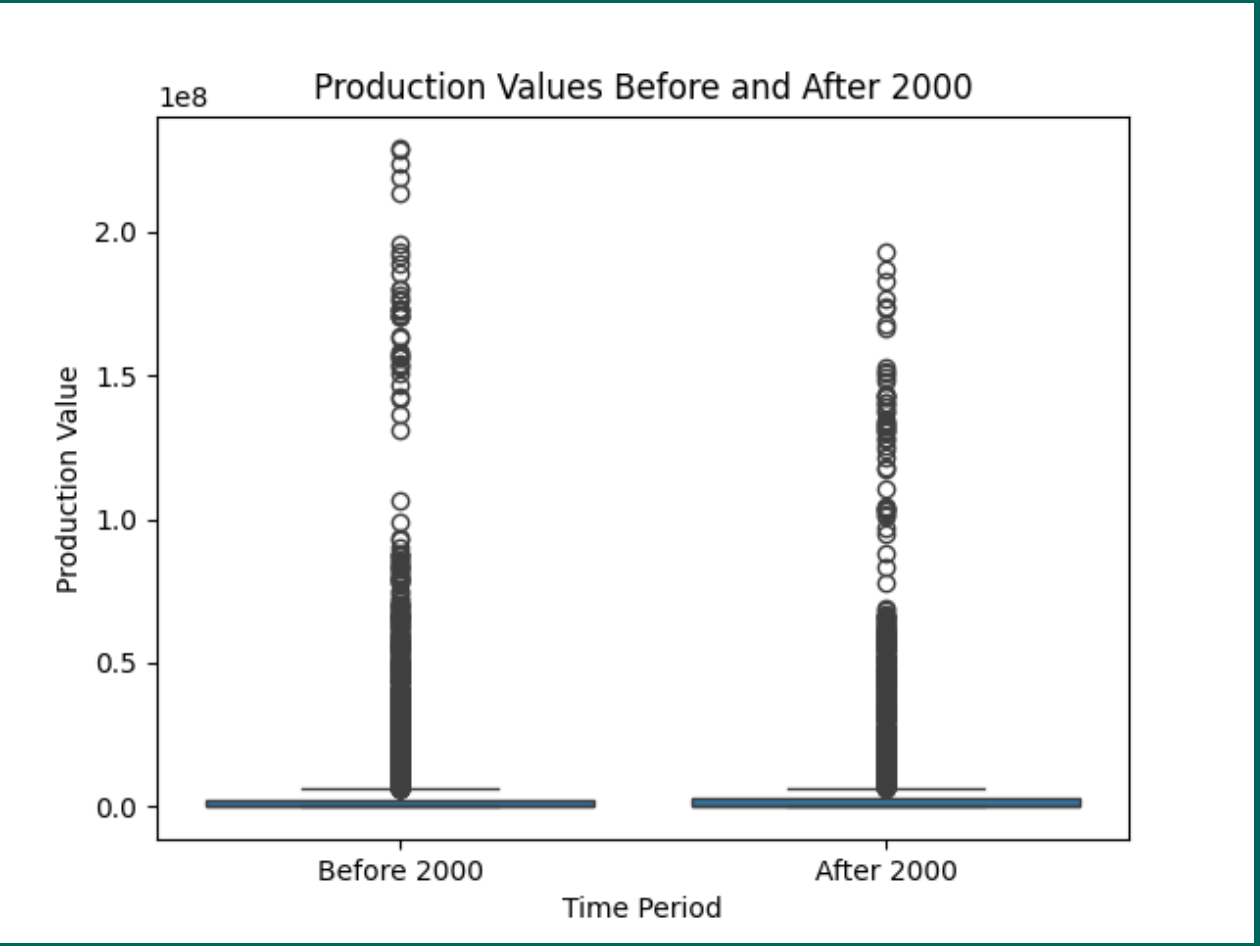
Scatter Plot With Regression Lines

- The scatterplot shows a **positive upward trend**, indicating that production values tend to increase over time.
- The fitted **regression line** forms this relationship and supports the results of the **correlation** & **regression** analysis, which shows a **positive relation** between **year** and **production levels**.



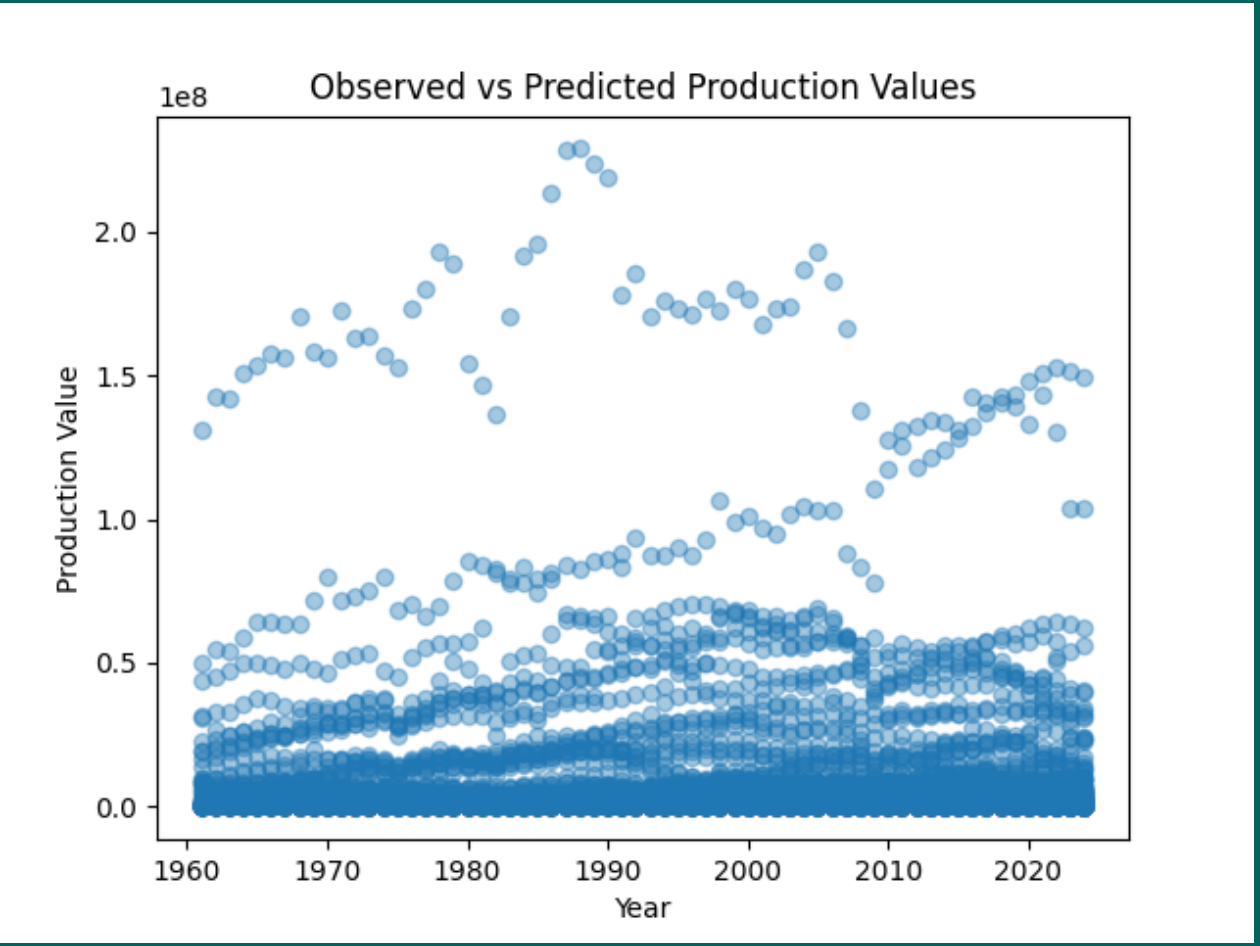
Boxplot

- The boxplot compares the distribution of production values across two time periods. The **median production value** appears **higher** after 2000's
- It supports the **hypothesis** test results indicating a big **increase** in **production levels** in more recent years.



Predicted Outcome Plot From Regression Model

- The predicted outcome plot compares observed production values with the regression model's predicted values.
- The fitted line illustrates the **overall increasing** trend in **production** over time
- Differences between observed points and the fitted line reflect unknown variation due to other factors not included in the model.



Thank You

Erik Jones
Jonese0804@students.bowiestate.edu